Name: Menna Hesham Ali Mohamed – Group: SE2

Lab 3 Handout Containerization and Cluster Orchestration

Task A1 - Start an interactive container:

```
PS C:\Users\Menna> docker run --rm -it --name nsdemo ubuntu:22.04 bash
Unable to find image 'ubuntu:22.04' locally
22.04: Pulling from library/ubuntu
f63eb04151bc: Pull complete
bf0d6867143e: Download complete
Digest: sha256:962f6cadeae0ea6284001009daa4cc9a8c37e75d1f5191cf0eb83fe565b63dd7
Status: Downloaded newer image for ubuntu:22.04
```

1. Hostname:

```
root@7de0704ce254:/# hostname
7de0704ce254
```

The hostname command returned a container-specific ID (**7de0704ce254**), which is different from the host machine's hostname. This shows that the container operates within its own UTS namespace, allowing it to have an independent hostname.

2. ps -ef:

```
root@7de0704ce254:/# ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 22:02 pts/0    00:00:00 bash
root          10       1  0 22:02 pts/0    00:00:00 ps -ef
```

The ps -ef output showed that the bash process inside the container is assigned PID 1. This indicates that the container has its own process namespace, where processes are isolated and managed independently from the host system.

3. ip addr:

```
root@7de0704ce254:/# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0@if8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 66:a5:36:3b:22:5f brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.17.0.2/16 brd 172.17.255.255 scope global eth0
       valid_lft forever preferred_lft forever
```

The ip addr command showed a network interface (eth0) with **IP address 172.17.0.2,** which belongs to Docker's internal bridge network. This confirms that the container has its own network namespace and does not directly share the host's network stack.

4. mount | head:

```
root@7de0704ce254:/# mount | head
overlay on / type overlay (rw,relatime,lowerdir=/var/lib/desktop-containerd/daemon/io.containerd.snapshotter.v1.overlayf
s/snapshots/59/fs:/var/lib/desktop-containerd/daemon/io.containerd.snapshotter.v1.overlayfs/snapshots/58/fs,upperdir=/va
r/lib/desktop-containerd/daemon/io.containerd.snapshotter.v1.overlayfs/snapshots/60/fs,workdir=/var/lib/desktop-containe
rd/daemon/io.containerd.snapshotter.v1.overlayfs/snapshots/60/work)
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
tmpfs on /dev type tmpfs (rw,nosuid,size=65536k,mode=755)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=620,ptmxmode=666)
sysfs on /sys type sysfs (ro,nosuid,nodev,noexec,relatime)
tmpfs on /sys/fs/cgroup type tmpfs (rw,nosuid,nodev,noexec,relatime,mode=755)
cgroup on /sys/fs/cgroup/cpuset type cgroup (ro,nosuid,nodev,noexec,relatime,cpuset)
cgroup on /sys/fs/cgroup/cpu type cgroup (ro,nosuid,nodev,noexec,relatime,cpu)
cgroup on /sys/fs/cgroup/cpuacct type cgroup (ro,nosuid,nodev,noexec,relatime,cpuacct)
cgroup on /sys/fs/cgroup/blkio type cgroup (ro,nosuid,nodev,noexec,relatime,blkio)
```

The mount output showed an overlay filesystem mounted on the root (/). This shows that the container uses a layered filesystem and has a separate view of the filesystem from the host.

5. cat /proc/1/cgroup

```
root@7de0704ce254:/# cat /proc/1/cgroup
15:name=systemd:/docker/7de0704ce254e0a07c84c0b933f6e90361d07a854c318a3deac3638739483c1d
14:misc:/docker/7de0704ce254e0a07c84c0b933f6e90361d07a854c318a3deac3638739483c1d
13:rdma:/docker/7de0704ce254e0a07c84c0b933f6e90361d07a854c318a3deac3638739483c1d
12:pids:/docker/7de0704ce254e0a07c84c0b933f6e90361d07a854c318a3deac3638739483c1d
11:hugetlb:/docker/7de0704ce254e0a07c84c0b933f6e90361d07a854c318a3deac3638739483c1d
10:net_prio:/docker/7de0704ce254e0a07c84c0b933f6e90361d07a854c318a3deac3638739483c1d
9:perf_event:/docker/7de0704ce254e0a07c84c0b933f6e90361d07a854c318a3deac3638739483c1d
8:net_cls:/docker/7de0704ce254e0a07c84c0b933f6e90361d07a854c318a3deac3638739483c1d
7:freezer:/docker/7de0704ce254e0a07c84c0b933f6e90361d07a854c318a3deac3638739483c1d
6:devices:/docker/7de0704ce254e0a07c84c0b933f6e90361d07a854c318a3deac3638739483c1d
5:memory:/docker/7de0704ce254e0a07c84c0b933f6e90361d07a854c318a3deac3638739483c1d
4:blkio:/docker/7de0704ce254e0a07c84c0b933f6e90361d07a854c318a3deac3638739483c1d
3:cpuacct:/docker/7de0704ce254e0a07c84c0b933f6e90361d07a854c318a3deac3638739483c1d
2:cpu:/docker/7de0704ce254e0a07c84c0b933f6e90361d07a854c318a3deac3638739483c1d
1:cpuset:/docker/7de0704ce254e0a07c84c0b933f6e90361d07a854c318a3deac3638739483c1d
0::/docker/7de0704ce254e0a07c84c0b933f6e90361d07a854c318a3deac3638739483c1d
```

The /proc/1/cgroup output showed multiple control groups associated with the container. This confirms that Docker uses cgroups to manage and limit resource usage such as CPU, memory, and process counts.

- **Conclusion:**

The experiment demonstrates that containers are isolated using Linux namespaces, which control what a process can see, while cgroups control how much of system resources a process can use.

Task A2 - Compare host and container process views:

```
C:\Users\Menna>docker ps
CONTAINER ID   IMAGE          COMMAND   CREATED          STATUS          PORTS     NAMES
7de0704ce254   ubuntu:22.04   "bash"    25 minutes ago   Up 25 minutes             nsdemo

C:\Users\Menna>docker inspect nsdemo --format "{{.State.Pid}}"
558

C:\Users\Menna>tasklist | findstr 558
svchost.exe                   5588 Services                   0      8,068 K
```

The container reports its main process (bash) as PID 1, while the host system assigns it a different PID (558 in this case). This difference occurs due to the use of the **PID namespace**.

Question: Why does the container think its main process is PID 1 while the host sees a different PID? Explain the role of the PID namespace.

A PID namespace isolates the process ID numbering inside the container from the host system. As a result:

- The container has its own independent process tree
- The first process inside the container always appears as PID 1
- The host system still tracks the process using its global PID

This allows containers to run processes independently without interfering with or being aware of processes on the host system.

Task A3 - Apply resource limits with cgroups:

```
PS D:\CMC_labs\lab3> docker run --rm -it --cpus="0.5" --memory="256m" ubuntu:22.04 bash
```

```
root@57a0ec7b29d7:/# stress --cpu 2 --vm 1 --vm-bytes 220M --timeout 30
stress: info: [236] dispatching hogs: 2 cpu, 0 io, 1 vm, 0 hdd
stress: info: [236] successful run completed in 30s
```

```
CONTAINER ID   NAME             CPU %     MEM USAGE / LIMIT   MEM %     NET I/O          BLOCK I/O   PIDS
57a0ec7b29d7   amazing_bouman   0.00%     18.8MiB / 256MiB    7.34%     50MB / 1.29MB    0B / 0B     1
7de0704ce254   nsdemo           0.00%     3MiB / 7.701GiB     0.04%     51.7MB / 1.3MB   0B / 0B     1
```

Question: How do cgroups complement namespaces?

Namespaces provide isolation by restricting what a container can see, such as processes, network interfaces, and filesystem. However, namespaces alone do not control how much of a resource a container can use. This is where **cgroups (control groups)** are used.

In this task:

- The container was limited to **0.5 CPU and 256 MB memory**
- The stress tool attempted to exceed these limits
- Using docker stats, it was observed that the container stayed within the defined resource boundaries

This demonstrates that:

- Namespaces isolate resources
- cgroups limit and control resource usage

Part B - Docker image layering: (B1 & B2)

```
PS D:\CMC_labs\lab3> docker history lab3-basic
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
e5d1f4089736   11 seconds ago   CMD ["python" "app.py"]                         0B        buildkit.dockerfile.v0
<missing>      11 seconds ago   EXPOSE [5000/tcp]                               0B        buildkit.dockerfile.v0
<missing>      11 seconds ago   COPY app.py . # buildkit                        12.3kB    buildkit.dockerfile.v0
<missing>      11 seconds ago   RUN /bin/sh -c pip install -r requirements.t…   16.5MB    buildkit.dockerfile.v0
<missing>      20 seconds ago   COPY requirements.txt . # buildkit              12.3kB    buildkit.dockerfile.v0
<missing>      20 seconds ago   WORKDIR /app                                    8.19kB    buildkit.dockerfile.v0
```

```
PS D:\CMC_labs\lab3> docker image ls
                                                                                                                  i Info →   U  In Use
IMAGE                                                                                   ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest                                                                      f9078146db2e       25.9kB         9.49kB    U
kindest/node:v1.35.0                                                                    4613778f3cfc       1.35GB          397MB
kindest/node@sha256:452d707d4862f52530247495d180205e029056831160e22870e37e3f6c1ac31f
                                                                                        452d707d4862        956MB         2.73kB
lab3-basic:latest                                                                       e5d1f4089736        199MB           49MB
ubuntu:22.04                                                                            962f6cadeae0        119MB         31.7MB    U
```

The docker history command shows that the Docker image is built in layers, where each instruction in the Dockerfile creates a new layer.

Task B3 - Build a multi-stage image:

```
PS D:\CMC_labs\lab3> docker image ls
                                                                                                                  i Info →   U  In Use
IMAGE                                                                                   ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest                                                                      f9078146db2e       25.9kB         9.49kB    U
kindest/node:v1.35.0                                                                    4613778f3cfc       1.35GB          397MB
kindest/node@sha256:452d707d4862f52530247495d180205e029056831160e22870e37e3f6c1ac31f
                                                                                        452d707d4862        956MB         2.73kB
lab3-basic:latest                                                                       e5d1f4089736        199MB           49MB
lab3-multi:latest                                                                       9615350bcd71        184MB         44.7MB
ubuntu:22.04                                                                            962f6cadeae0        119MB         31.7MB    U
PS D:\CMC_labs\lab3> docker history lab3-multi
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
9615350bcd71   18 seconds ago   CMD ["python" "app.py"]                         0B        buildkit.dockerfile.v0
<missing>      18 seconds ago   EXPOSE [5000/tcp]                               0B        buildkit.dockerfile.v0
<missing>      18 seconds ago   COPY app.py . # buildkit                        12.3kB    buildkit.dockerfile.v0
<missing>      18 seconds ago   COPY /install /usr/local # buildkit             5.51MB    buildkit.dockerfile.v0
<missing>      7 minutes ago    WORKDIR /app                                    8.19kB    buildkit.dockerfile.v0
```

Questions: Which image is smaller? Which steps are cached if only app.py changes? Why does layer order matter?

Two images were built:

- lab3-basic: 199MB
- lab3-multi: 184MB

The multi-stage image is smaller because it excludes unnecessary build dependencies and intermediate files.

Layer Differences:

- In the basic image:
  - Dependencies are installed directly in the final image
  - All build artifacts remain
- In the multi-stage image:
  - Dependencies are installed in a separate **builder stage**
  - Only the required output (/install) is copied into the final image
  - Unnecessary files are discarded

Layer Order Matters because placing COPY requirements.txt before COPY app.py ensures that:

- Dependencies are only reinstalled if requirements change
- Application changes do not trigger expensive rebuilds

Part C - Local Kubernetes orchestration with kind: C1

```
PS C:\Users\PC> kind create cluster --name lab3-cluster --image kindest/node:v1.29.2
Creating cluster "lab3-cluster" ...
 ✓ Ensuring node image (kindest/node:v1.29.2) 🖼
 ✓ Preparing nodes 📦
 ✓ Writing configuration 📜
 ✓ Starting control-plane 🕹️
 ✓ Installing CNI 🔌
 ✓ Installing StorageClass 💾
Set kubectl context to "kind-lab3-cluster"
You can now use your cluster with:

kubectl cluster-info --context kind-lab3-cluster

Have a question, bug, or feature request? Let us know! https://kind.sigs.k8s.io/#community 😊
PS C:\Users\PC> kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:51680
CoreDNS is running at https://127.0.0.1:51680/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
PS C:\Users\PC> kubectl get nodes -o wide
NAME                       STATUS   ROLES           AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE
           KERNEL-VERSION                      CONTAINER-RUNTIME
lab3-cluster-control-plane   Ready    control-plane   40s   v1.29.2   172.18.0.2    <none>        Debian GNU/Linux 12 (b
ookworm)   5.15.167.4-microsoft-standard-WSL2   containerd://1.7.13
```

A local Kubernetes cluster was successfully created using kind. The cluster consists of a single node running inside Docker. The node is in Ready state, meaning it can schedule Pods.

Task C2 - Load your image into kind:

```
PS C:\Users\PC> docker image ls
                                                                                                  i Info →   U  In Use
IMAGE                                                                          ID             DISK USAGE   CONTENT SIZE   EXTRA
kindest/node:v1.29.2                                                           51a1434a5397       1.39GB          409MB    U
kindest/node@sha256:452d707d4862f52530247495d180205e029056831160e22870e37e3f6c1ac31f
                                                                               452d707d4862       1.35GB          397MB
lab3-basic:latest                                                              46180fa35c33        199MB           49MB
lab3-multi:latest                                                              c93ef8b6b020        184MB         44.7MB
ubuntu:22.04                                                                   962f6cadeae0        119MB         31.7MB    U
PS C:\Users\PC> kind load docker-image lab3-basic --name lab3-cluster
Image: "lab3-basic" with ID "sha256:46180fa35c337c7982669cd2ad07173c252b5d4349c9673318e362ea49890e52" not yet present on
 node "lab3-cluster-control-plane", loading...
PS C:\Users\PC> kind load docker-image lab3-multi --name lab3-cluster
Image: "lab3-multi" with ID "sha256:c93ef8b6b020e93d802baedcd424343ff2624927742e5faead261780d2979cdc" not yet present on
 node "lab3-cluster-control-plane", loading...
```

The Docker images were successfully loaded into the kind Kubernetes cluster. Since kind runs Kubernetes inside Docker containers, images must be explicitly loaded into the cluster node to be available for deployments.

Task C3 - Deploy a replicated application:

```
PS D:\CMC_labs\lab3> kubectl apply -f deployment.yaml
deployment.apps/lab3-web created
PS D:\CMC_labs\lab3> kubectl get pods -o wide
NAME                        READY   STATUS    RESTARTS   AGE   IP           NODE                       NOMINATED NODE
  READINESS GATES
lab3-web-5c5f649ddc-hsf9f   1/1     Running   0          10s   10.244.0.6   lab3-cluster-control-plane   <none>
  <none>
lab3-web-5c5f649ddc-lcv65   1/1     Running   0          10s   10.244.0.5   lab3-cluster-control-plane   <none>
  <none>
lab3-web-5c5f649ddc-rbbhl   1/1     Running   0          10s   10.244.0.7   lab3-cluster-control-plane   <none>
  <none>
```

The Kubernetes Deployment created 3 identical Pod replicas. The system automatically scheduled them on the available node without manual intervention. All Pods are running the same container image.

```
PS D:\CMC_labs\lab3> kubectl get deployments
NAME       READY   UP-TO-DATE   AVAILABLE   AGE
lab3-web   3/3     3            3           4m39s
PS D:\CMC_labs\lab3> kubectl get pods
NAME                        READY   STATUS    RESTARTS   AGE
lab3-web-5c5f649ddc-hsf9f   1/1     Running   0          4m46s
lab3-web-5c5f649ddc-lcv65   1/1     Running   0          4m46s
lab3-web-5c5f649ddc-rbbhl   1/1     Running   0          4m46s
PS D:\CMC_labs\lab3> kubectl get services
NAME           TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
kubernetes     ClusterIP   10.96.0.1      <none>        443/TCP   22m
lab3-web-svc   ClusterIP   10.96.78.250   <none>        80/TCP    26s
```

- **Deployment:** Defines the desired state of the application, such as how many replicas should run, and ensures Kubernetes maintains that state automatically.
- **Pod:** The smallest Kubernetes unit that runs one or more containers and represents a running instance of the application.
- **Service:** Provides a stable network endpoint to access Pods and routes traffic to them even if Pods are replaced or moved.

The Deployment defines the desired state (3 replicas of the app). Kubernetes ensures these Pods are running. The Service provides a stable networking endpoint that routes traffic to the Pods using label selectors.

Task C4 - Access the service:

```
PS D:\CMC_labs\lab3> curl http://localhost:8080/


StatusCode        : 200
StatusDescription : OK
Content           : Hello from Lab 3
RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 16
                    Content-Type: text/html; charset=utf-8
                    Date: Sun, 26 Apr 2026 03:42:07 GMT
                    Server: Werkzeug/3.1.8 Python/3.12.13

                    Hello from Lab 3
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 16], [Content-Type, text/html; charset=utf-8], [Date, Sun,
                    26 Apr 2026 03:42:07 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 16
```

```
PS D:\CMC_labs\lab3> curl http://localhost:8080/health


StatusCode        : 200
StatusDescription : OK
Content           : ok
RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 2
                    Content-Type: text/html; charset=utf-8
                    Date: Sun, 26 Apr 2026 03:42:19 GMT
                    Server: Werkzeug/3.1.8 Python/3.12.13

                    ok
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 2], [Content-Type, text/html; charset=utf-8], [Date, Sun,
                    26 Apr 2026 03:42:19 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 2
```

Observation:
The Flask application was successfully accessed using Kubernetes port-forwarding. The Service correctly routed requests to running Pods, and both the root and health endpoints responded with HTTP 200, confirming correct deployment and networking.

8. Part D - Scheduling and placement: D1

```
PS D:\CMC_labs\lab3> kubectl label nodes --all node-role=general
node/lab3-cluster-control-plane labeled
PS D:\CMC_labs\lab3> kubectl get nodes --show-labels
NAME                        STATUS   ROLES           AGE   VERSION   LABELS
lab3-cluster-control-plane   Ready    control-plane   36m   v1.29.2   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/o
s=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=lab3-cluster-control-plane,kubernetes.io/os=linux,node-role.kube
rnetes.io/control-plane=,node-role=general
```

The node lab3-cluster-control-plane was successfully labeled with node-role=general.

Task D2 - Force placement with a node selector

```
PS D:\CMC_labs\lab3> kubectl apply -f deployment.yaml
deployment.apps/lab3-web configured
PS D:\CMC_labs\lab3> kubectl get pods -o wide
NAME                        READY   STATUS        RESTARTS   AGE   IP            NODE                         NOMINATED
NODE   READINESS GATES
lab3-web-585d88dd9c-5fpjz   1/1     Running       0          8s    10.244.0.8    lab3-cluster-control-plane   <none>
       <none>
lab3-web-585d88dd9c-ddgfd   1/1     Running       0          8s    10.244.0.9    lab3-cluster-control-plane   <none>
       <none>
lab3-web-585d88dd9c-f5bl9   1/1     Running       0          7s    10.244.0.10   lab3-cluster-control-plane   <none>
       <none>
lab3-web-5c5f649ddc-hsf9f   1/1     Terminating   0          24m   10.244.0.6    lab3-cluster-control-plane   <none>
       <none>
lab3-web-5c5f649ddc-lcv65   1/1     Terminating   0          24m   10.244.0.5    lab3-cluster-control-plane   <none>
       <none>
lab3-web-5c5f649ddc-rbbhl   1/1     Terminating   0          24m   10.244.0.7    lab3-cluster-control-plane   <none>
       <none>
```

- After adding the nodeSelector, Kubernetes created new Pods that follow the updated scheduling rule.
- Old Pods were gradually terminated and replaced.
- This demonstrates Kubernetes' **rolling update mechanism**.

Why is this different from hard-coding a machine name in traditional deployment models?

Hard-coding a machine name tightly couples the application to a specific node, reducing flexibility and resilience. If that node fails, the application cannot be scheduled elsewhere.

In contrast, nodeSelector uses labels to describe *requirements*, not specific machines. Any node that satisfies the label can run the workload, making the system more flexible, scalable, and fault-tolerant.

How does declarative scheduling improve resilience and maintainability?

Declarative scheduling improves resilience and maintainability by allowing developers to specify *what* the system should achieve rather than *how* to achieve it.

Instead of manually assigning workloads to specific machines, Kubernetes uses high-level constraints (such as labels and selectors) to automatically determine placement. This means:

- **Resilience improves** because workloads are not tied to a single node. If a node fails, Kubernetes can reschedule Pods on any other node that satisfies the declared conditions.
- **Maintainability improves** because configurations are simpler and more flexible. Changes can be made by updating the desired state (e.g., modifying labels or selectors) without rewriting deployment logic.
- The system becomes **self-managing**, reducing manual intervention and operational errors.

Part E - Self-healing and health probes:

```
PS D:\CMC_labs\lab3> kubectl get pods
NAME                        READY   STATUS    RESTARTS   AGE
lab3-web-585d88dd9c-5fpjz   1/1     Running   0          8m12s
lab3-web-585d88dd9c-ddgfd   1/1     Running   0          8m12s
lab3-web-585d88dd9c-f5bl9   1/1     Running   0          8m11s
PS D:\CMC_labs\lab3> kubectl delete pod lab3-web-585d88dd9c-5fpjz
pod "lab3-web-585d88dd9c-5fpjz" deleted from default namespace
PS D:\CMC_labs\lab3> kubectl get pods -w
NAME                        READY   STATUS    RESTARTS   AGE
lab3-web-585d88dd9c-ddgfd   1/1     Running   0          9m10s
lab3-web-585d88dd9c-f5bl9   1/1     Running   0          9m9s
lab3-web-585d88dd9c-j4mfv   1/1     Running   0          35s
```

When a Pod is deleted, Kubernetes automatically creates a replacement. This happens because the Deployment maintains the desired number of replicas.

Why is this called *reconciliation* rather than simple restart logic?

This process is called reconciliation because Kubernetes continuously compares the **actual state** of the system with the **desired state** defined in the Deployment.

When a Pod is deleted, Kubernetes does not simply restart that exact Pod. Instead, it detects that the number of running Pods is lower than the desired number and creates a completely new Pod to restore the correct state.

Unlike traditional restart logic, which focuses on restarting a failed process, reconciliation is a continuous control loop that ensures the system always matches the declared configuration.

Task E2 - Add readiness and liveness probes:

```
PS D:\CMC_labs\lab3> kubectl apply -f probe-deployment.yaml
deployment.apps/lab3-web-probe created
PS D:\CMC_labs\lab3> kubectl get pods
NAME                              READY   STATUS    RESTARTS   AGE
lab3-web-585d88dd9c-ddgfd         1/1     Running   0          14m
lab3-web-585d88dd9c-f5bl9         1/1     Running   0          14m
lab3-web-585d88dd9c-j4mfv         1/1     Running   0          6m7s
lab3-web-probe-f5dc5585b-zssdk    1/1     Running   0          9s
lab3-web-probe-f5dc5585b-zz926    1/1     Running   0          9s
```

A new Deployment with readiness and liveness probes was successfully created. Two new Pods (lab3-web-probe-*) were launched.

Task E3 - Observe what happens if the container process dies:

```
PS D:\CMC_labs\lab3> kubectl delete pod lab3-web-probe-f5dc5585b-zssdk
pod "lab3-web-probe-f5dc5585b-zssdk" deleted from default namespace
```

```
PS D:\CMC_labs\lab3> kubectl get pods -w
NAME                              READY   STATUS    RESTARTS   AGE
lab3-web-585d88dd9c-ddgfd         1/1     Running   0          21m
lab3-web-585d88dd9c-f5bl9         1/1     Running   0          21m
lab3-web-585d88dd9c-j4mfv         1/1     Running   0          12m
lab3-web-probe-f5dc5585b-zssdk    1/1     Running   0          6m40s
lab3-web-probe-f5dc5585b-zz926    1/1     Running   0          6m40s
lab3-web-probe-f5dc5585b-zssdk    1/1     Terminating   0          8m56s
lab3-web-probe-f5dc5585b-4dkbg    0/1     Pending       0          0s
lab3-web-probe-f5dc5585b-4dkbg    0/1     Pending       0          0s
lab3-web-probe-f5dc5585b-4dkbg    0/1     ContainerCreating   0          0s
lab3-web-probe-f5dc5585b-4dkbg    0/1     Running             0          1s
lab3-web-probe-f5dc5585b-4dkbg    1/1     Running             0          5s
```

Kubernetes created a new Pod instead of restarting the existing container.

When the original Pod was terminated, the Deployment controller detected that the number of running replicas had fallen below the desired state. It then triggered the creation of a new Pod, which was scheduled and started automatically.

Reflection questions:

1. Why do namespaces alone not guarantee fair resource use?

Namespaces only provide isolation of system views (processes, network, filesystem), but they do not limit resource consumption. A container can still consume all CPU or memory unless restricted by cgroups.

2. How do cgroups improve cluster stability?

Cgroups enforce resource limits such as CPU and memory per container. This prevents resource exhaustion and ensures fair sharing between workloads, improving overall cluster stability.

3. Why is Docker image layering important for large-scale orchestration?

Layering allows reuse of unchanged parts of images, reducing build time, storage usage, and network transfer costs. In large-scale systems, this significantly improves deployment efficiency and scalability.

4. What does Kubernetes mean by desired state?

Desired state is the configuration defined in Kubernetes objects (e.g., Deployments). Kubernetes continuously works to ensure the actual system state matches this desired configuration.

5. How is self-healing different from traditional manual operations?

Self-healing is automatic and continuous. Kubernetes detects failures and restores the system without human intervention, unlike traditional systems where administrators manually restart or fix services.

6. Why are readiness and liveness probes not interchangeable?

Readiness probes control whether a Pod receives traffic, while liveness probes determine whether a container should be restarted. Using them interchangeably can lead to incorrect restarts or traffic routing to unhealthy services.

7. Limitation of single-node kind cluster?

A single-node cluster cannot demonstrate real distributed scheduling, failover, or load balancing across multiple nodes. It only simulates orchestration behavior on one machine.